

Maximum Likelihood Estimation and Logistic Regression

Computational Mathematics and Statistics

Jason Bryer, Ph.D.

March 24, 2026

Data Project

Please see

spring2026.dav5300.net/assignments/project.html

for details on the project proposal.

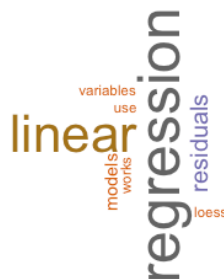
Please sign-up for a time to meet with me (via Zoom) on April 14th to discuss your project. Please have your proposal ready (template on the course website) and come prepared with questions.



Scan the QR code or [click here](http://spring2026.dav5300.net/assignments/project.html) to sign-up

One Minute Paper Results

What was the most important thing you learned during this class?



What important question remains unanswered for you?

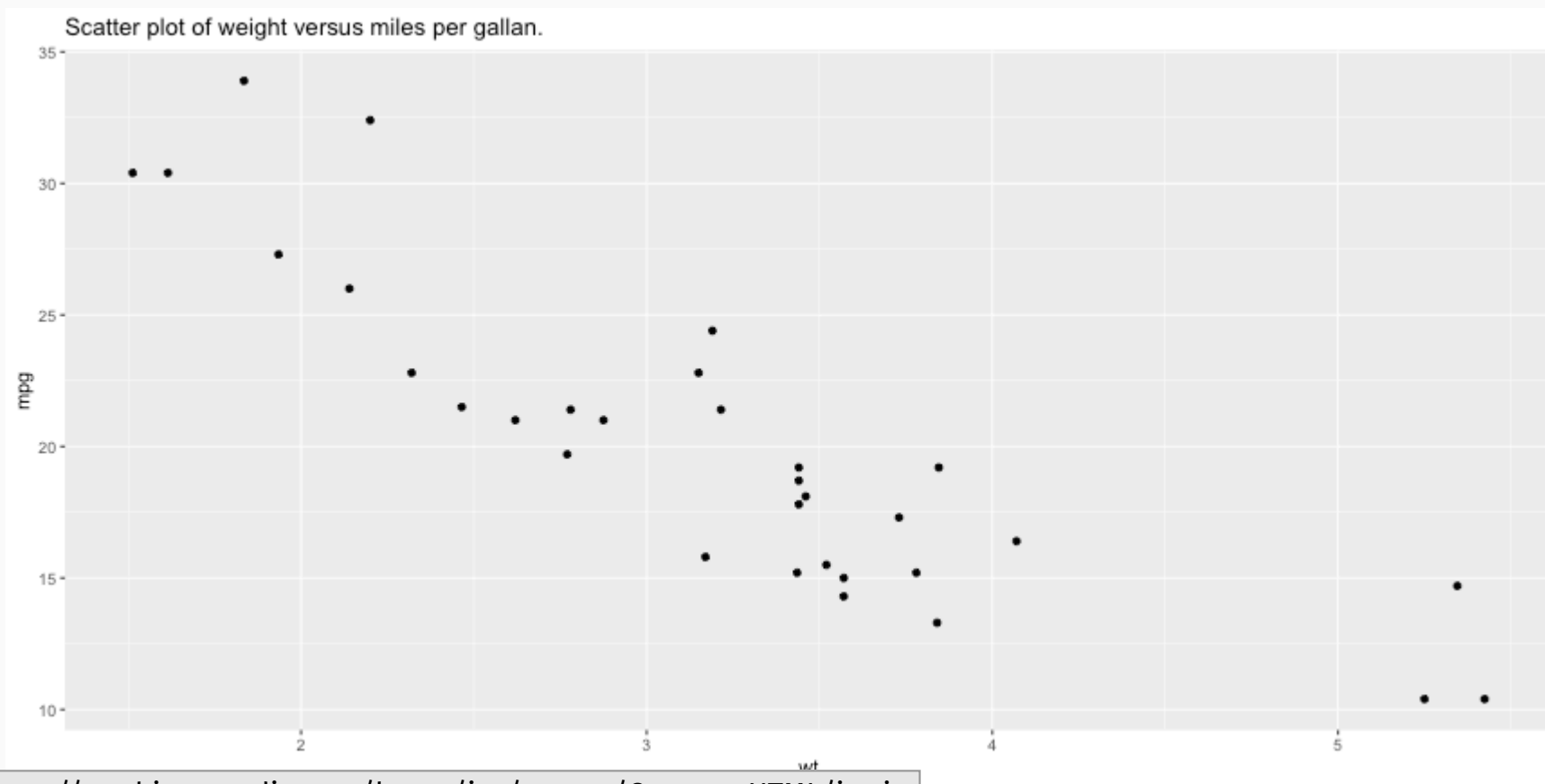


Maximum Likelihood Estimation

Maximum Likelihood Estimation (MLE) is an important procedure for estimating parameters in statistical models. It is often first encountered when modeling a dichotomous outcome variable vis-à-vis logistic regression. However, it is the backbone of **generalized linear models** (GLM) which allow for error distribution models other than the normal distribution. Most introductions to MLE rely on mathematical notation that for many students is opaque and hinders learning how this method works. The document outlines an approach to understanding MLE that relies on visualizations and mathematical notation is only used when necessary.

Bivariate Regression

We will begin with a typical bivariate regression using the `mtcars` data set where we wish to predict `mpg` (miles per gallon) from `wt` (weight in 1,000 lbs).



File failed to load: <http://mathjax.rstudio.com/latest/jax/output/CommonHTML/jax.js>

Linear Regression

Our goal is to estimate

β_1

where β_1 is the slope and β_0 is the intercept.

Ordinary Least Squares

With ordinary least squares (OLS) regression our goal is to minimize the residual sum of squares (RSS):

y_i

where y_i is the variable to be predicted, $\hat{y}_i$ is the predicted value of y_i , and n is the sample size.

The basic properties we know about regression are:

- The correlation measures the strength of the relationship between x and y (see [this shiny app](#) for an excellent visual overview of correlations).
- The correlation ranges between -1 and 1.
- The mean of x and y must fall on the line.
- The slope of a line is defined as the change in y over the change in x ($\frac{\Delta y}{\Delta x}$). For regression use the ratio of the standard deviations such that the correlation is defined as $r = \frac{s_y}{s_x} \beta$ where β is the slope, r is the correlation, and s_x is the sample standard deviation.

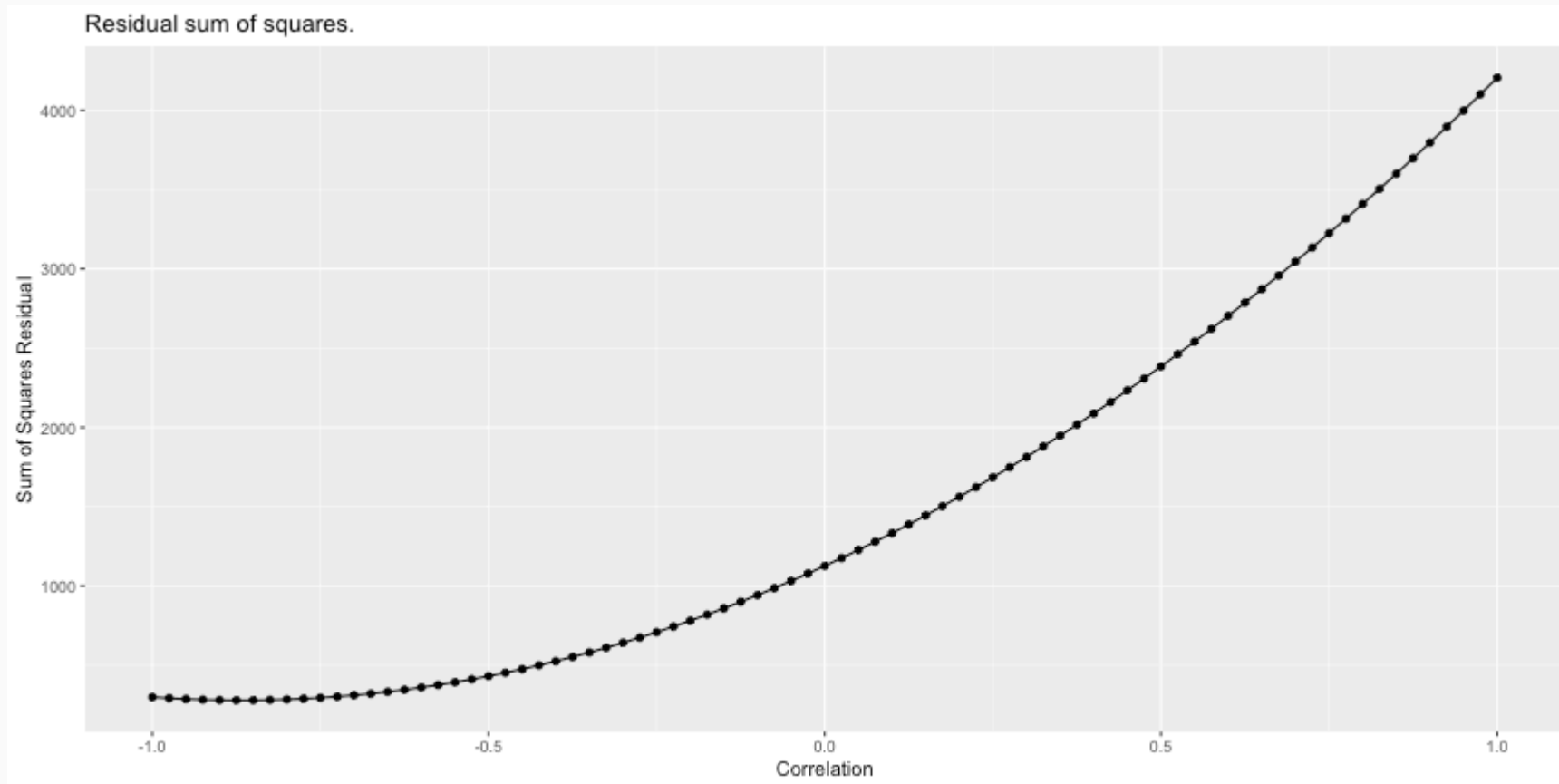
Ordinary Least Squares

We can easily calculate the RSS for various correlations (r) ranging between -1 and 1.

```
y <- mtcars$mpg
x <- mtcars$wt
mean.y <- mean(y)
mean.x <- mean(x)
sd.y <- sd(y)
sd.x <- sd(x)
ols <- tibble(
  r = seq(-1, 1, by = 0.025),          # Correlation
  m = r * (sd.y / sd.x),               # Slope
  b = mean.y - m * mean.x              # Intercept
) %>% rowwise() %>%
  mutate(ss = sum((y - (m * x + b))^2)) %>% # Sum of squares residuals
  as.data.frame()
```


Ordinary Least Squares

```
ggplot(ols, aes(x = r, y = ss)) + geom_path() + geom_point() +  
  ggtitle('Residual sum of squares.') + xlab('Correlation') + ylab('Sum of Squares Residual')
```



File failed to load: <http://mathjax.rstudio.com/latest/jax/output/CommonHTML/jax.js>

Ordinary Least Squares

The correlation that resulted in the smallest RSS is -0.875.

```
ols %>% dplyr::filter(ss == min(ss)) # Select the row with the smallest RSS
```

```
##           r           m           b           ss  
## 1 -0.875 -5.389687 37.4306 278.3826
```

Calculating the correlation in R gives us -0.8676594 and the slope is -5.3444716 which is close to our estimate here. We could get a more accurate result if we tried smaller steps in the correlation (see the `by` parameter in the `seq` function above).

Minimizing RSS Algorithmically

This approach works well here because the correlation is bounded between -1 and 1 and we can easily calculate the RSS for a bunch of possible correlations. However, there are more efficient ways of finding the correlation that minimizes the RSS than trying correlations equally distributed across the possible range. For example, consider the following simple algorithm:

1. Calculate the RSS for *[Math Processing Error]*.
2. Calculate the RSS for *[Math Processing Error]* If *[Math Processing Error]* then calculate the RSS with *[Math Processing Error]*, else calculate the RSS with *[Math Processing Error]*

We can repeat this procedure, essentially halving the distance in each iteration until we find a sufficiently small RSS.

Minimizing RSS Algorithmically

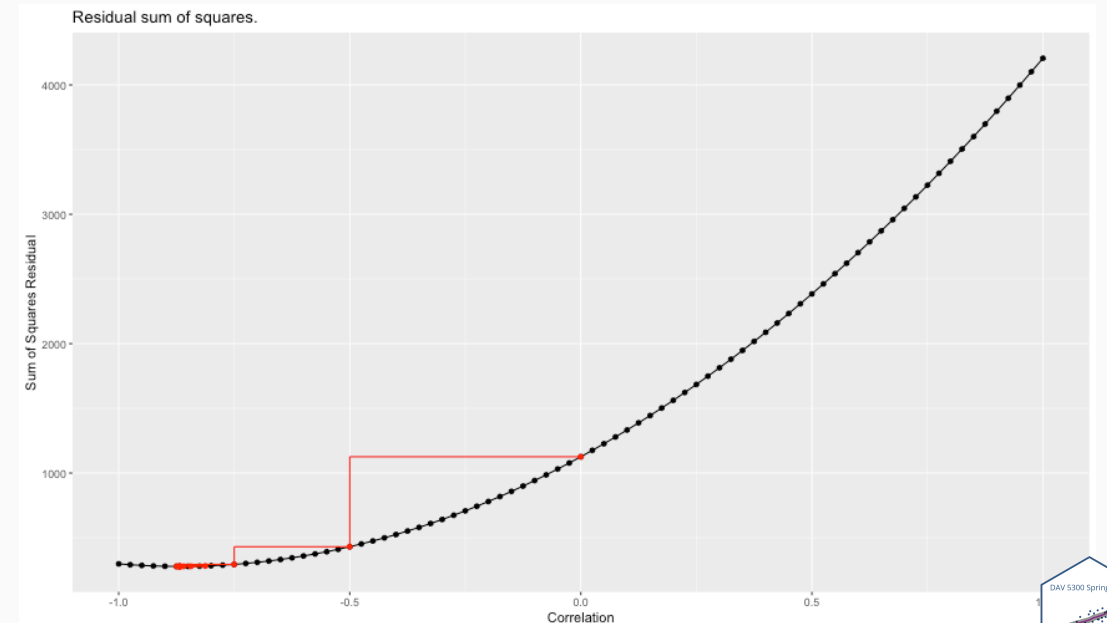
```
y <- mtcars$mpg
x <- mtcars$wt
ssr <- function(r, x, y) {
  mean.y <- mean(y); mean.x <- mean(x)
  sd.y <- sd(y); sd.x <- sd(x)
  m = r * (sd.y / sd.x)
  b = mean.y - m * mean.x
  ss = sum((y - (m * x + b))^2)
  return(ss)
}
r_left <- -1
r_right <- 1
ssr_left <- ssr(r_left, x = x, y = y)
ssr_right <- ssr(r_right, x = x, y = y)
iter <- numeric()
threshold <- 0.00001
while(abs(ssr_left - ssr_right) > threshold) {
  if(ssr_left < ssr_right) {
    r_right <- r_right - (r_right - r_left) / 2
    ssr_right <- ssr(r_right, x = x, y = y)
    iter <- c(iter, r_right)
  } else {
    r_left <- r_left + (r_right - r_left) / 2
    ssr_left <- ssr(r_left, x = x, y = y)
    iter <- c(iter, r_left)
  }
}
```

```
r_left; r_right; cor(x, y)
```

```
## [1] -0.8676758
```

```
## [1] -0.8676147
```

```
## [1] -0.8676594
```



The `optim` function

This process is, in essence, the idea of numerical optimization procedures. In R, the `optim` function implements the **Nedler-Mead** (Nedler & Mead, 1965) and **Limited Memory BFGS** (Byrd et al, 1995) methods for optimizing a set of parameters. The former is the default but we will use the latter throughout this document since it allows for specifying bounds for certain parameters (e.g. only consider positive values). The details of *how* the algorithm works is beyond the scope of this article (see this **interactive tutorial** by Ben Frederickson for a good introduction), instead we will focus on *what* the algorithm does.

Example

To begin, we must define a function that calculates a metric for which the optimizer is going to minimize (or maximize).

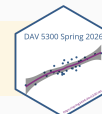
```
residual_sum_squares <- function(parameters, predictor, outcome) {  
  a <- parameters[1] # Intercept  
  b <- parameters[2] # beta coefficient  
  predicted <- a + b * predictor  
  residuals <- outcome - predicted  
  ss <- sum(residuals^2)  
  return(ss)  
}
```

The `parameters` is a vector of the parameters the algorithm is going to minimize (or maximize). Here, these will be the slope and intercept. The `predictor` and `outcome` are parameters passed through from the `...` parameter on the `optim` function and are necessary for us to calculate the RSS. We can now get the RSS for any set of parameters.

```
residual_sum_squares(c(37, -5), mtcars$wt, mtcars$mpg)
```

File failed to load: <http://mathjax.rstudio.com/latest/jax/output/CommonHTML/jax.js>

```
## [1] 303.5247
```



Small Digression: Saving the steps along the way...

In order to explore each step of the algorithm, we need to wrap the `optim` function to capture the parameters and output of the function. The `optim_save` function will add two elements to the returned list: `iterations` is the raw list of the parameters and output saved and `iterations_df` is a `data.frame` containing the same data.

```
optim_save <- function(par, fn, ...) {  
  iterations <- list()  
  wrap_fun <- function(parameters, ...) {  
    n <- length(iterations)  
    result <- fn(parameters, ...)  
    iterations[[n + 1]] <- c(parameters, result)  
    return(result)  
  }  
  optim_out <- stats::optim(par, wrap_fun, ...)  
  optim_out$iterations <- iterations  
  optim_out$iterations_df <- as.data.frame(do.call(rbind, iterations))  
  names(optim_out$iterations_df) <- c(paste0('Param', 1:length(par)), 'Result')  
  optim_out$iterations_df$Iteration <- 1:nrow(optim_out$iterations_df)  
  return(optim_out)  
}
```

OLS with the `optim` function

We can now call the `optim_save` function with our `residual_sum_squares` function. We initialize the algorithm with two random values for the intercept and slope, respectively. Note that we are using Broyden, Fletcher, Goldfarb, and Shanno optimization method which allows for the specification of bounds on the parameter estimates which we will use later.

```
optim.rss <- optim_save(  
  par = runif(2),  
  fn = residual_sum_squares,  
  method = "L-BFGS-B",  
  predictor = mtcars$wt,  
  outcome = mtcars$mpg  
)
```


OLS with the `optim` function

The `par` parameter provides the final parameter estimates.

```
optim.rss$par
```

```
## [1] 37.285114 -5.344469
```

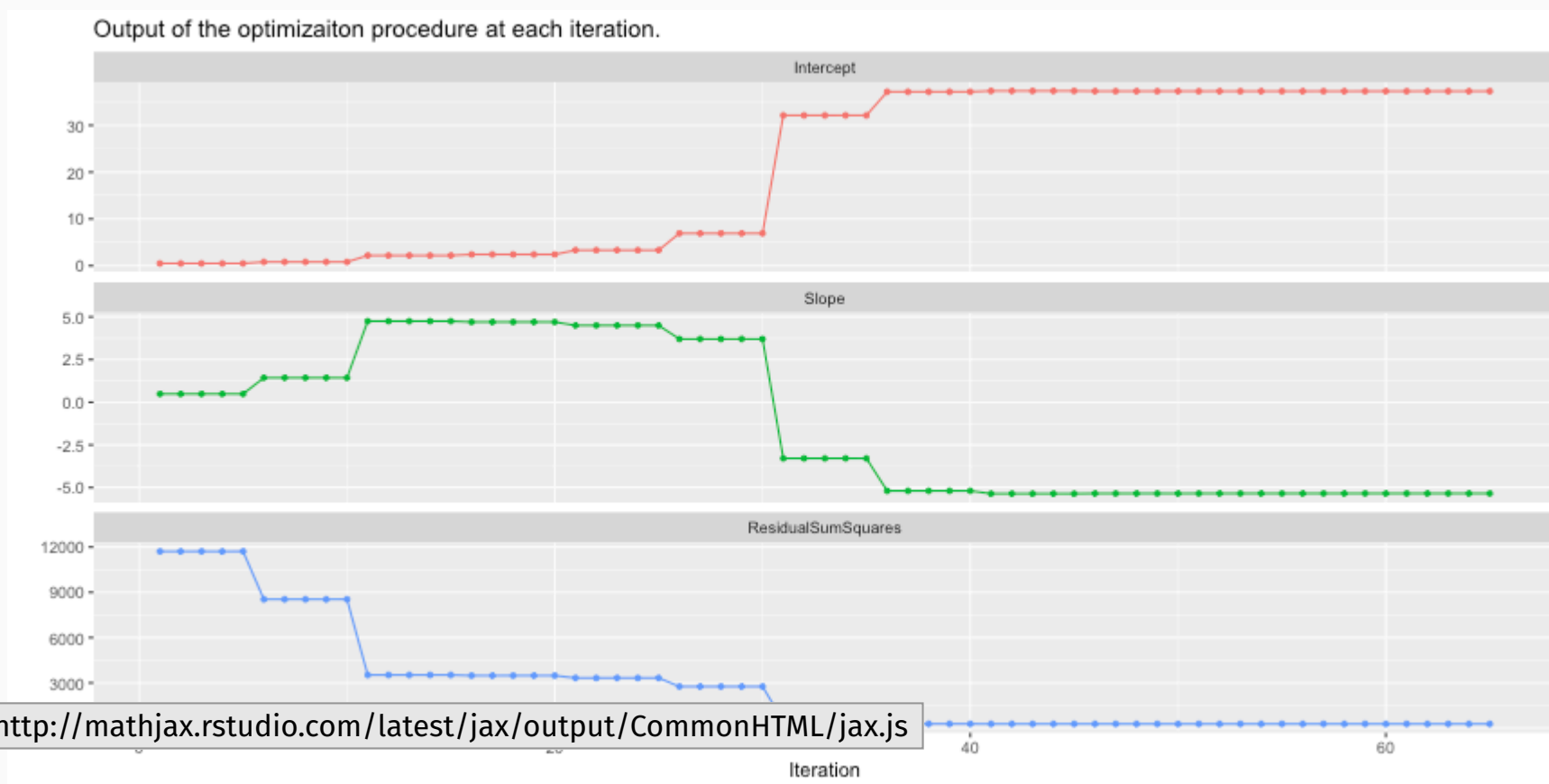
We can see that the parameters are accurate to at least four decimal places to the OLS method used by the `lm` function.

```
lm.out <- lm(mpg ~ wt, data = mtcars)
lm.out$coefficients
```

```
## (Intercept)          wt
##  37.285126    -5.344472
```

OLS with the `optim` function

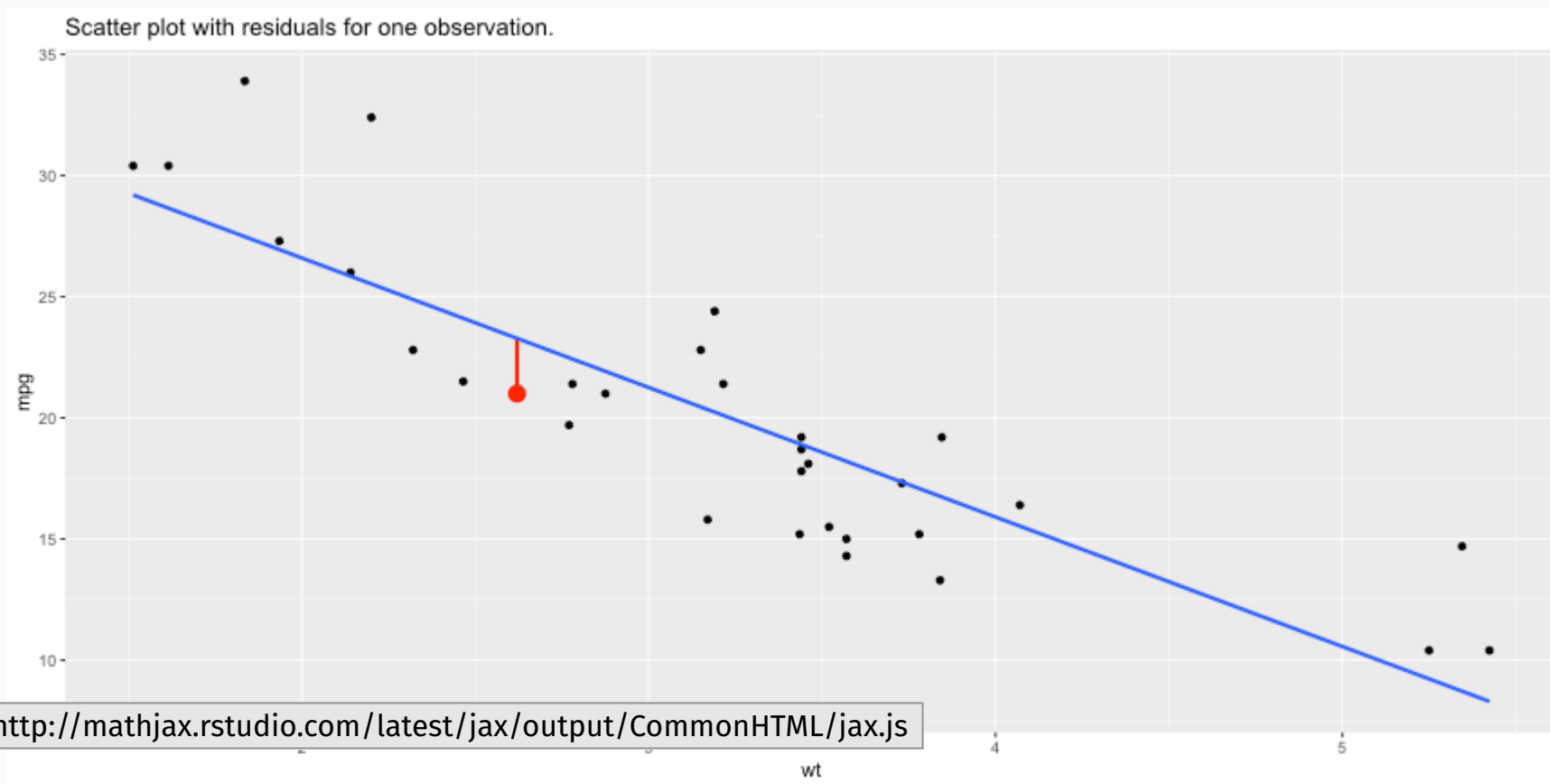
It took the `optim` function 65 iterations to find the optimal set of parameters that minimized the RSS. This figure shows the value of the parameters (i.e. intercept and slope) and the RSS for each iteration.



File failed to load: <http://mathjax.rstudio.com/latest/jax/output/CommonHTML/jax.js>

Residuals to Likelihoods

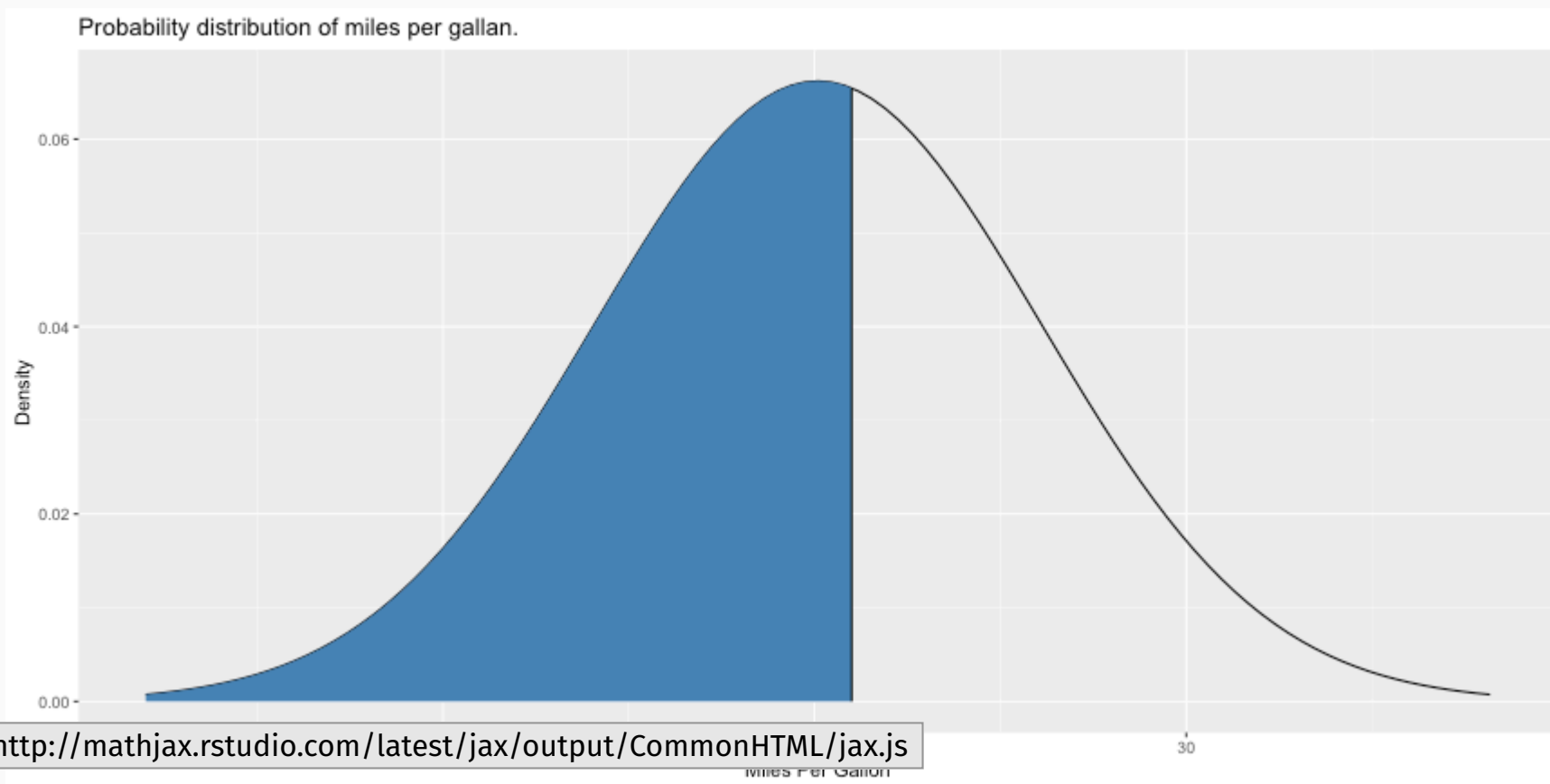
Now that we have laid the groundwork for finding parameters algorithmically, we need to introduce another way of evaluating how well parameters *fit* the data, namely the likelihood. First, let's revisit what we are doing in OLS.



File failed to load: <http://mathjax.rstudio.com/latest/jax/output/CommonHTML/jax.js>

Probability

We often think of probabilities as the areas under a fixed distribution. For example, the first car in `mtcars` is Mazda RX4 with an average miles per gallon of 21 and weighs 2620lbs. The probability of a car with a miles per gallon less than Mazda RX4 given the data we have in `mtcars` is 0.5599667.



File failed to load: <http://mathjax.rstudio.com/latest/jax/output/CommonHTML/jax.js>

Probabilities and Likelihoods

For probabilities, we are working with a fixed distribution, that is:

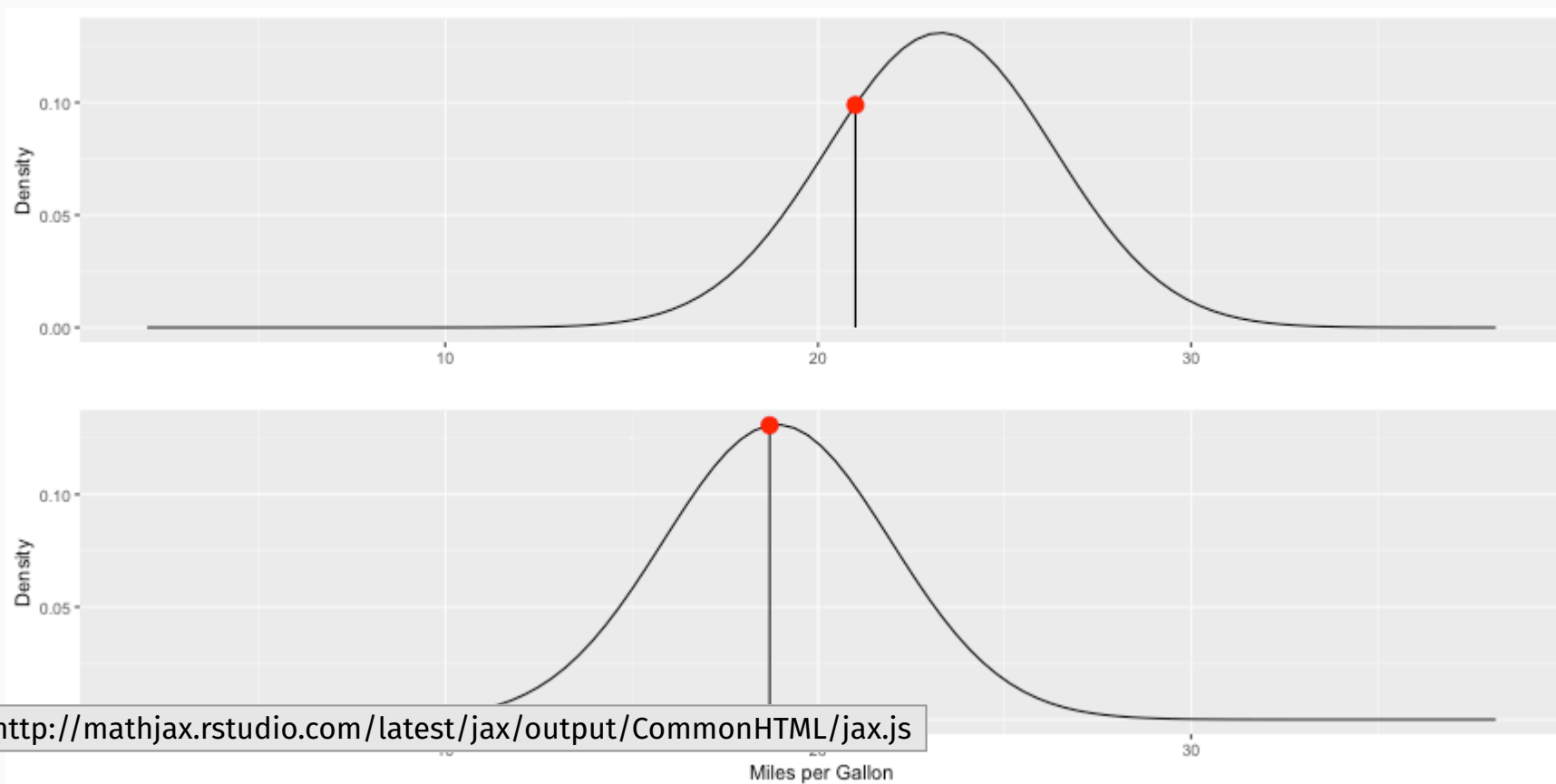
[Math Processing Error]

The likelihood are the y-axis values (i.e. density) for fixed data points with distributions that can move, that is:

[Math Processing Error]

Likelihoods

The likelihood is the height of the density function. This figure depicts two likelihood for two observations. The mean of each distribution is equal to *[Math Processing Error]* and the intercept (also known as the error term) defines the standard deviation of the distribution.



File failed to load: <http://mathjax.rstudio.com/latest/jax/output/CommonHTML/jax.js>

Log-Likelihood Function

We can then calculate the likelihood for each observation in our data. Unlike OLS, we now want to *maximize* the sum of these values. Also, we are going to use the log of the likelihood so we can add them instead of multiplying. We can now define our log likelihood function:

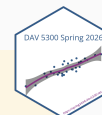
```
loglikelihood <- function(parameters, predictor, outcome) {  
  a <- parameters[1]      # intercept  
  b <- parameters[2]      # slope / beta coefficient  
  sigma <- parameters[3] # error  
  ll.vec <- dnorm(outcome, mean = a + b * predictor, sd = sigma, log = TRUE)  
  return(sum(ll.vec))  
}
```

Note that we have to estimate a third parameter, sigma, which is the error term and defines the standard deviation for the normal distribution for estimating the likelihood. This is connected to the distribution of the residuals as we will see later. We can now calculate the log-likelihood for any combination of parameters.

```
loglikelihood(c(37, -5, sd(mtcars$mpg)), predictor = mtcars$wt, outcome = mtcars$mpg)
```

File failed to load: <http://mathjax.rstudio.com/latest/jax/output/CommonHTML/jax.js>

```
## [1] -91.06374
```



Maximum Likelihood Estimation

We can now use the `optim_save` function to find the parameters that *maximize* the log-likelihood. Note two important parameter changes:

1. We are specifying the `lower` parameter so that the algorithm will not try negative values for sigma since the variance cannot be negative.
2. The value for the `control` parameter indicates that we wish to maximize the values instead of minimizing (which is the default).

```
optim.ll <- optim_save(  
  runif(3),                # Random initial values  
  loglikelihood,           # Log-likelihood function  
  lower = c(-Inf, -Inf, 1.e-5), # The lower bounds for the values, note sigma, cannot be negative  
  method = "L-BFGS-B",  
  control = list(fnscale = -1), # Indicates that the maximum is desired rather than the minimum  
  predictor = mtcars$wt,  
  outcome = mtcars$mpg  
)
```


Maximum Likelihood Estimation

We can get our results and compare them to the results of the `lm` function and find that they match to at least four decimal places.

```
optim.ll$par[1:2]
```

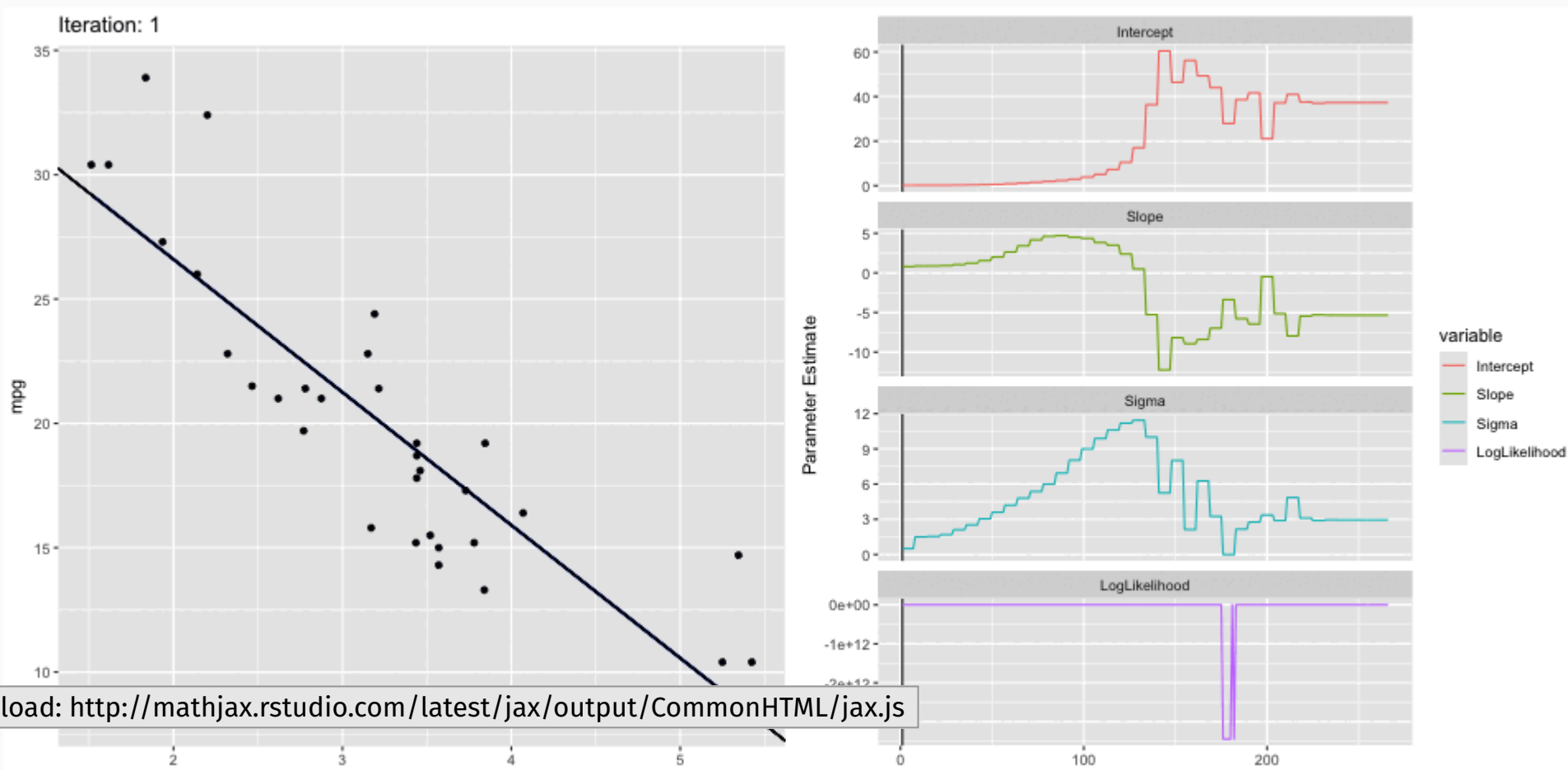
```
## [1] 37.28512 -5.34447
```

```
lm.out$coefficients
```

```
## (Intercept)      wt  
##  37.285126   -5.344472
```

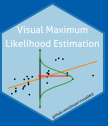
The steps of MLE

This figure shows the estimated regression line for each iteration of the optimization procedure (on the left; OLS regression line in blue; MLE regression line in black) with the estimated parameters and log-likelihood for all iterations on the right. This Shiny app will allow you to explore this process interactively: `shiny_demo('mle')`

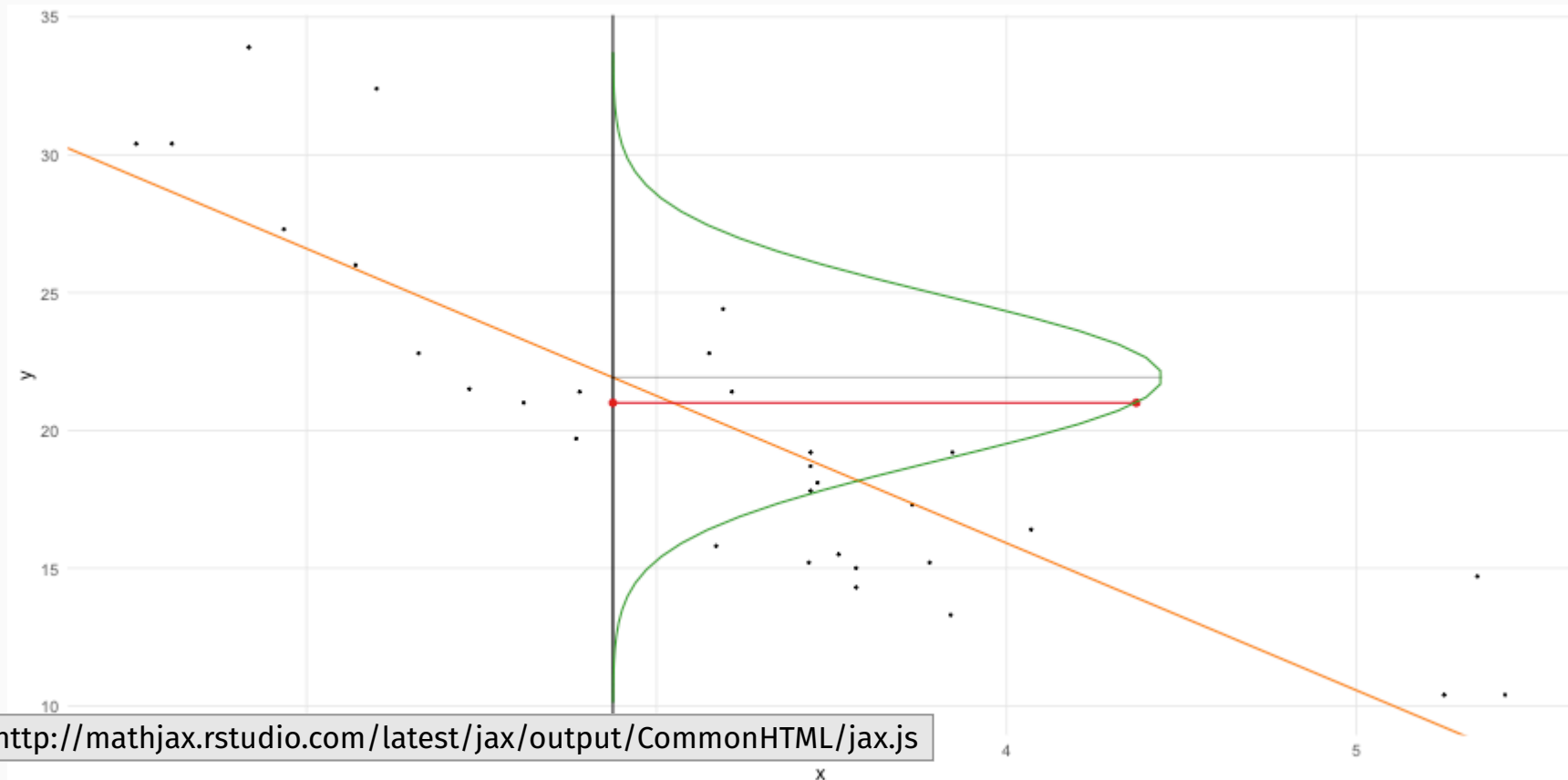


File failed to load: <http://mathjax.rstudio.com/latest/jax/output/CommonHTML/jax.js>

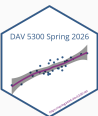
Likelihood Visualized



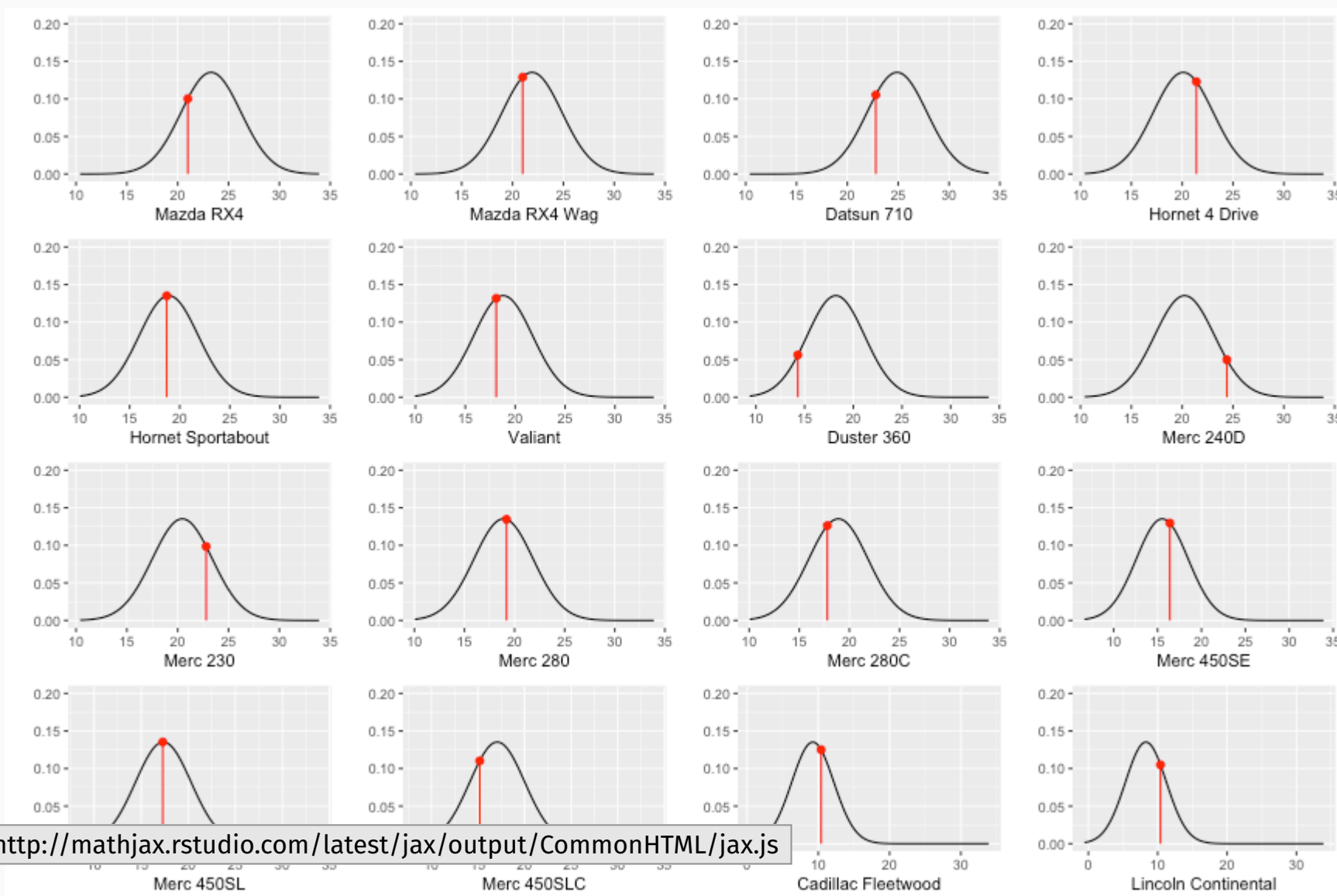
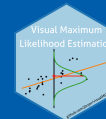
```
VisualStats::plot_likelihood(x = mtcars$wt, y = mtcars$mpg, pt = 2,  
                             intercept = optim.ll$par[1],  
                             slope = optim.ll$par[2],  
                             sigma = optim.ll$par[3])
```



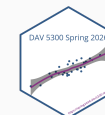
File failed to load: <http://mathjax.rstudio.com/latest/jax/output/CommonHTML/jax.js>



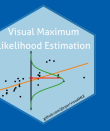
Likelihood Visualized



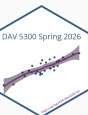
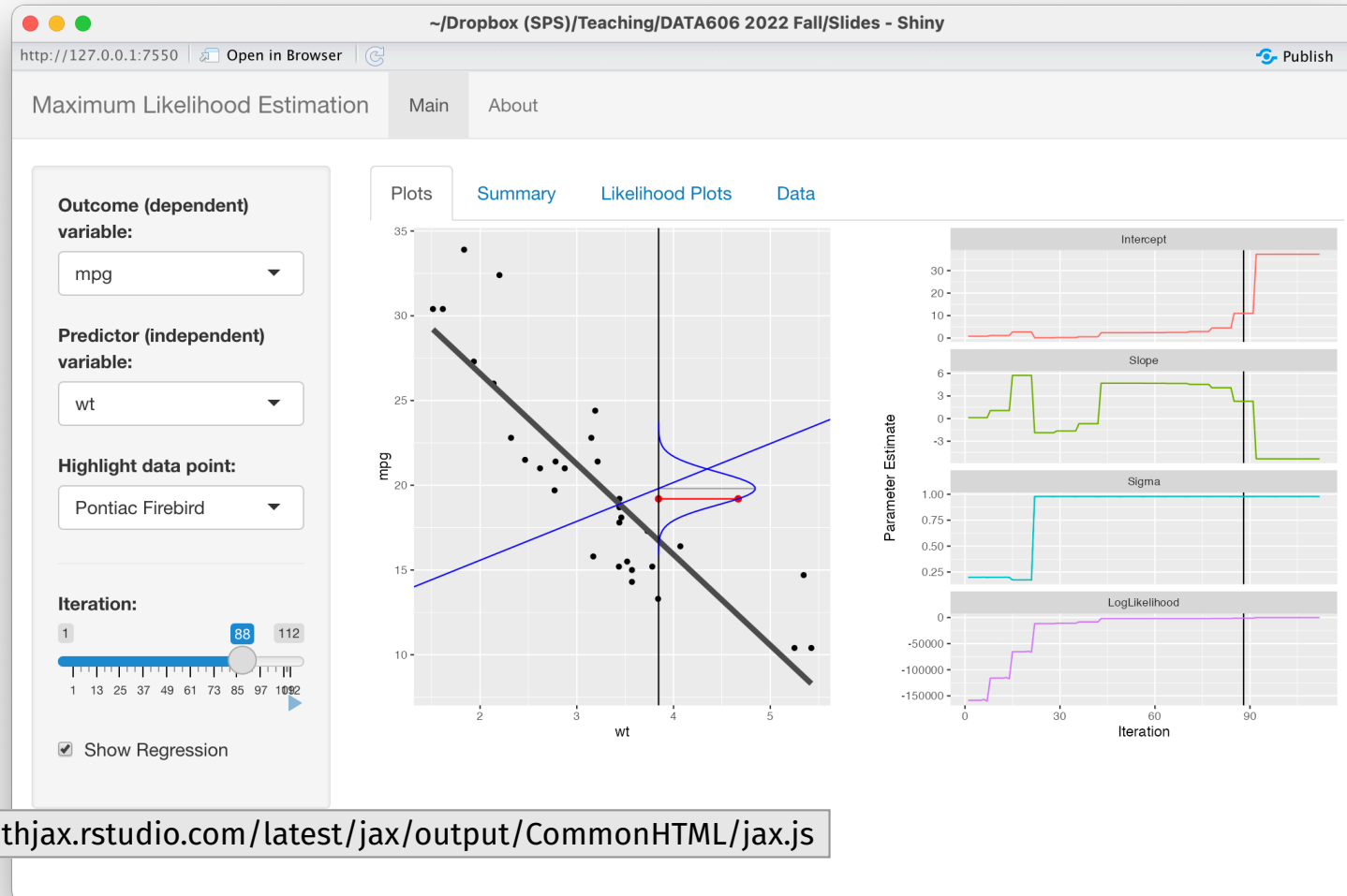
File failed to load: <http://mathjax.rstudio.com/latest/jax/output/CommonHTML/jax.js>



Likelihood Visualized



```
remotes::install_github('jbryer/VisualStats')  
VisualStats::mle_shiny()
```



Root-Mean-Square Error

With MLE we need to estimate what is often referred to as the error term, or as we saw above is the standard deviation of the normal distribution from which we are estimating the likelihood from. In the previous figure notice that the normal distribution is drawn vertically. This is because the likelihood is estimated from the error, or the residuals. In OLS we often report the root-mean-square deviation (RMSD, or root-mean-square error, RMSE). The RMSD is the standard deviation of the residuals:

[Math Processing Error]

Where *[Math Processing Error]* is the observation, *[Math Processing Error]* is the observed value, *[Math Processing Error]* is the estimated (predicted) value, and *[Math Processing Error]* is the sample size. Below, we see that the numerical optimizer matches the RMSD within a rounding error.

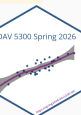
```
optim.ll$par[3]
```

```
## [1] 2.949163
```

```
sqrt(sum(resid(lm.out)^2) / nrow(mtcars))
```

```
## [1] 2.949163
```

File failed to load: <http://mathjax.rstudio.com/latest/jax/output/CommonHTML/jax.js>



Logistic Regression

Example: Hours Studying Predicting Passing

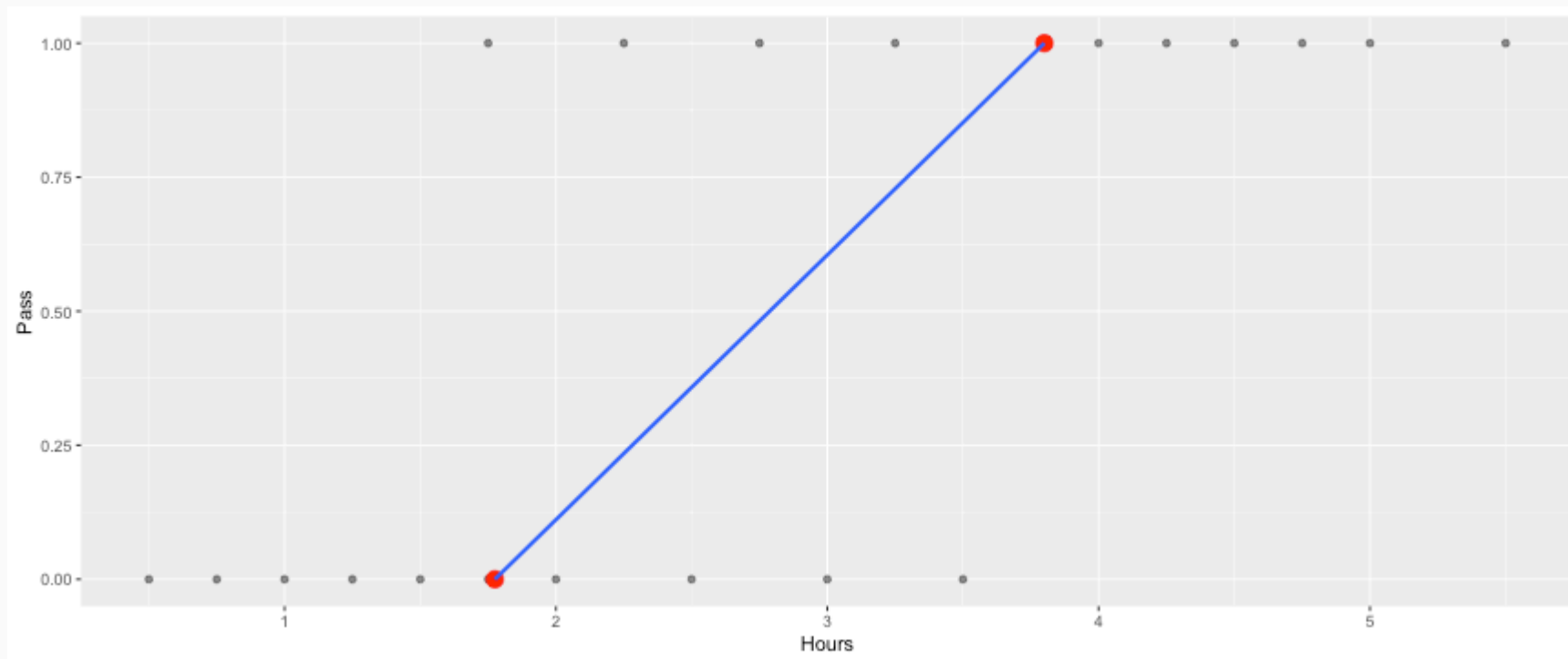
```
study <- data.frame(  
  Hours=c(0.50,0.75,1.00,1.25,1.50,1.75,1.75,2.00,2.25,2.50,2.75,3.00,  
          3.25,3.50,4.00,4.25,4.50,4.75,5.00,5.50),  
  Pass=c(0,0,0,0,0,0,0,1,0,1,0,1,0,1,0,1,1,1,1,1,1)  
)  
study[sample(nrow(study), 5),]
```

```
##      Hours Pass  
## 7      1.75    1  
## 12     3.00    0  
## 13     3.25    1  
## 14     3.50    0  
## 17     4.50    1
```

```
tab <- describeBy(study$Hours, group = study$Pass, mat = TRUE, skew = FALSE)  
tab$group1 <- as.integer(as.character(tab$group1))
```


Dichotomous (x) and continuous (y) variables

```
ggplot(study, aes(x = Pass, y = Hours)) + geom_point(alpha = 0.5) +  
  geom_point(data = tab, aes(x = group1, y = mean), color = 'red', size = 4) +  
  geom_smooth(method = lm, se = FALSE, formula = y ~ x) + coord_flip()
```

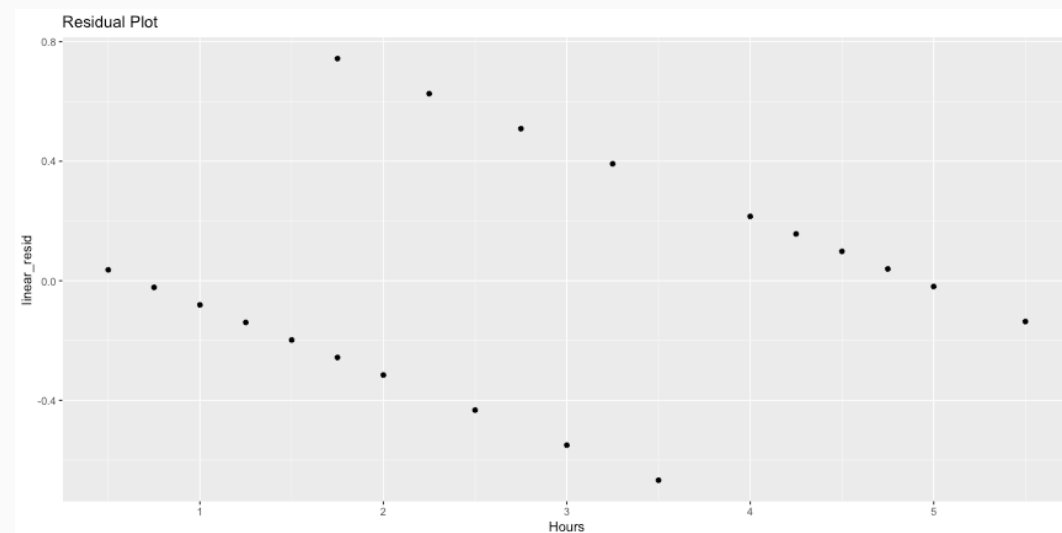


Ordinary Least Squares

```
lm.out <- lm(Pass ~ Hours, data = study)
summary(lm.out)
```

```
##
## Call:
## lm(formula = Pass ~ Hours, data = study)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.66715 -0.21262 -0.02053  0.17157  0.74339
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.15394    0.18315  -0.840  0.411655
## Hours        0.23460    0.05813   4.036  0.000775 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.3819 on 18 degrees of freedom
## Multiple R-squared:  0.4751,    Adjusted R-squared:
```

```
study$linear_resid <- resid(lm.out)
ggplot(study, aes(x = Hours, y = linear_resid)) +
  geom_point() +
  ggtitle('Residual Plot')
```



Regression so far...

At this point we have covered:

- Simple linear regression
 - Relationship between numerical response and a numerical or categorical predictor
- Multiple regression (next week)
 - Relationship between numerical response and multiple numerical and/or categorical predictors
- Maximum Likelihood Estimation

All of the approaches we have used so far have a quantitative variable with normally distributed errors (i.e. residuals).

What we haven't seen is what to do when the predictors are weird (nonlinear, complicated dependence structure, etc.) or when the response is weird (categorical, count data, etc.)

Odds

Odds are another way of quantifying the probability of an event, commonly used in gambling (and logistic regression).

For some event $[Math Processing Error]$,

$[Math Processing Error]$

Similarly, if we are told the odds of E are $[Math Processing Error]$ to $[Math Processing Error]$ then

$[Math Processing Error]$

which implies

$[Math Processing Error]$

Generalized Linear Models

Generalized linear models (GLM) are a generalization of OLS that allows for the response variables (i.e. dependent variables) to have an error distribution that is **not** distributed normally. All generalized linear models have the following three characteristics:

1. A probability distribution describing the outcome variable.
2. A linear model: *[Math Processing Error]*.
3. A link function that relates the linear model to the parameter of the outcome distribution: *[Math Processing Error]* or *[Math Processing Error]*.

We can estimate GLMs using maximum likelihood estimation (MLE). What will change is the log-likelihood function.

Logistic Regression

Logistic regression is a GLM used to model a binary categorical variable using numerical and categorical predictors.

We assume a binomial distribution produced the outcome variable and we therefore want to model p the probability of success for a given set of predictors.

To finish specifying the Logistic model we just need to establish a reasonable link function that connects *[Math Processing Error]* to *[Math Processing Error]*. There are a variety of options but the most commonly used is the logit function.

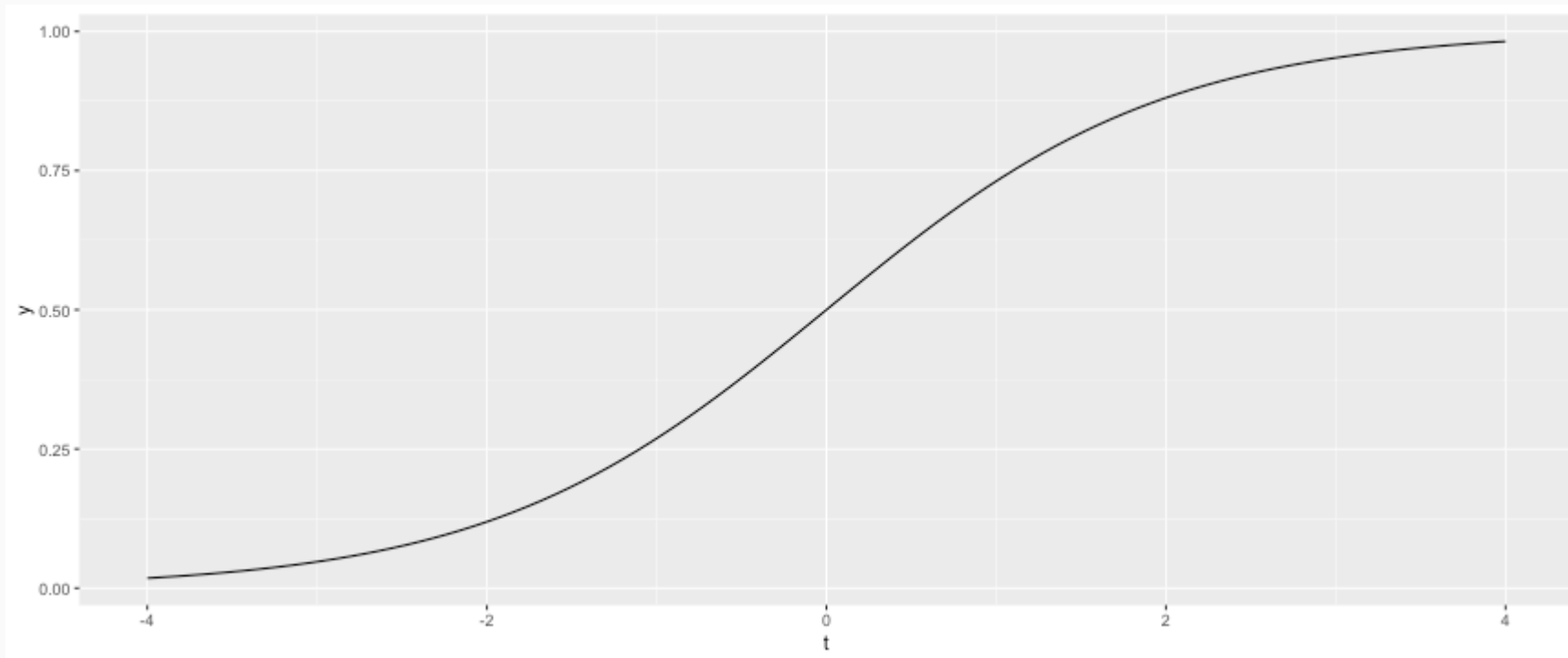
Logit function

[Math Processing Error]

The Logistic Function

[Math Processing Error]

```
logistic <- function(t) { return(1 / (1 + exp(-t))) }  
ggplot() + stat_function(fun = logistic, n = 101) + xlim(-4, 4) + xlab('t')
```



File failed to load: <http://mathjax.rstudio.com/latest/jax/output/CommonHTML/jax.js>

t as a Linear Function

[Math Processing Error]

The logistic function can now be rewritten as

[Math Processing Error]

Similar to OLS, we wish to minimize the errors. However, instead of minimizing the least squared residuals, we will use a maximum likelihood function.

Loglikelihood Function

We need to define logit function and the log-likelihood function that will be used by the optim function. Instead of using the normal distribution as above (using the dnorm function), we are using a binomial distribution and the logit to link the linear combination of predictors.

```
logit <- function(x, beta0, beta1) {  
  return( 1 / (1 + exp(-beta0 - beta1 * x)) )  
}  
  
loglikelihood.binomial <- function(parameters, predictor, outcome) {  
  a <- parameters[1] # Intercept  
  b <- parameters[2] # beta coefficient  
  p <- logit(predictor, a, b)  
  ll <- sum( outcome * log(p) + (1 - outcome) * log(1 - p))  
  return(ll)  
}
```

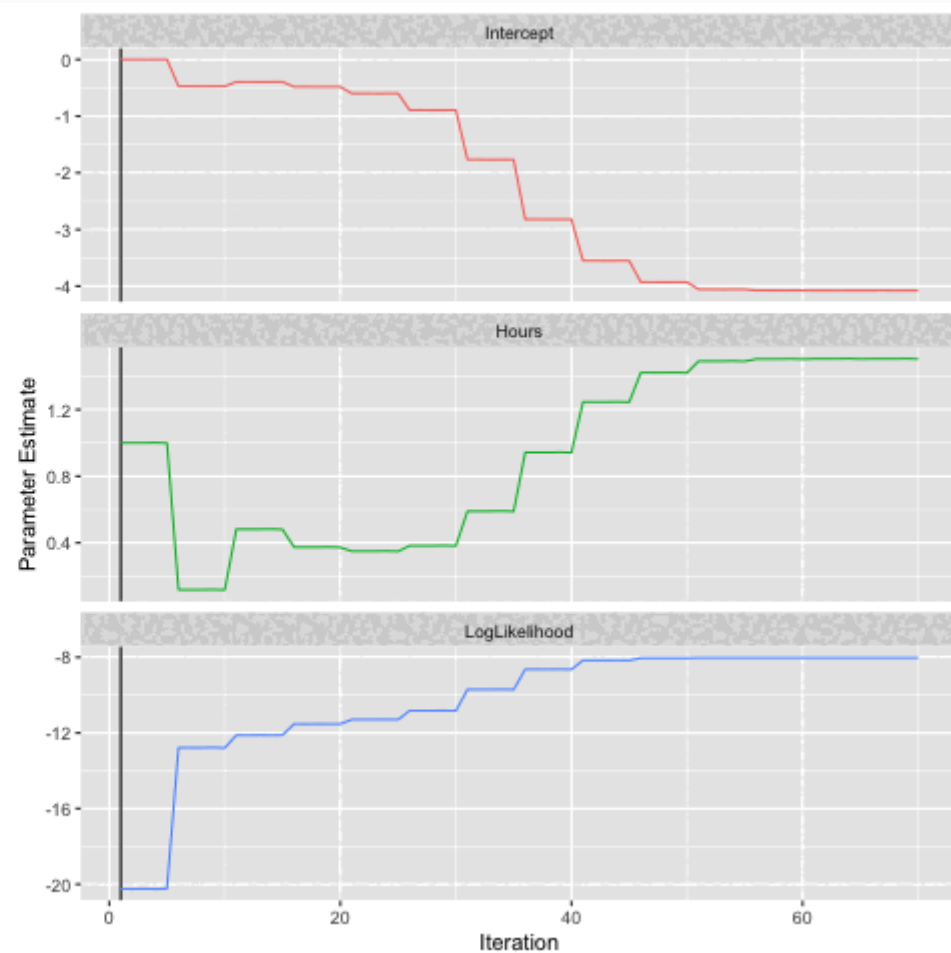
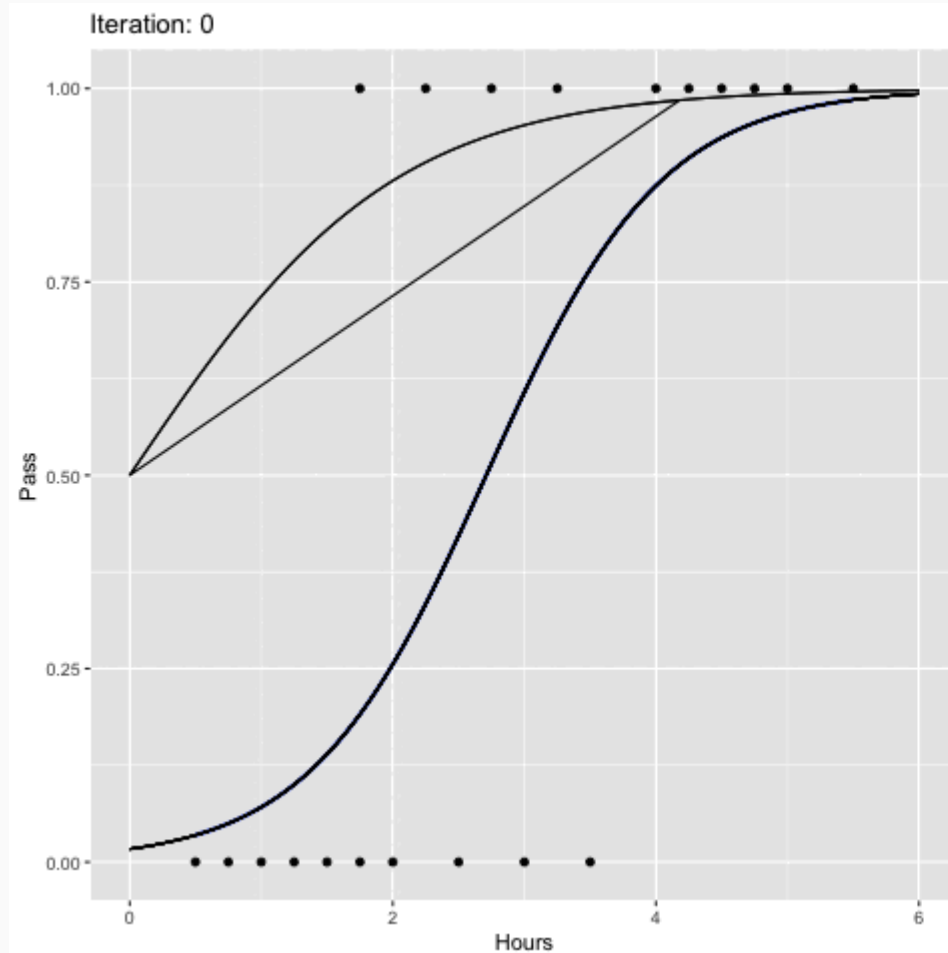
Estimating parameters using the `optim` function

```
optim.binomial <- optim_save(  
  c(0, 1), # Initial values  
  loglikelihood.binomial,  
  method = "L-BFGS-B",  
  control = list(fnscale = -1),  
  predictor = study$Hours,  
  outcome = study$Pass  
)
```

```
optim.binomial$par
```

```
## [1] -4.077575  1.504624
```

How did the optimizer get to this result?



The glm function

```
( lr.out <- glm(Pass ~ Hours, data = study, family = binomial(link = 'logit')) )
```

```
##  
## Call:  glm(formula = Pass ~ Hours, family = binomial(link = "logit"),  
##      data = study)  
##  
## Coefficients:  
## (Intercept)      Hours  
##      -4.078      1.505  
##  
## Degrees of Freedom: 19 Total (i.e. Null);  18 Residual  
## Null Deviance:      27.73  
## Residual Deviance: 16.06      AIC: 20.06
```

How does this compare to the `optim` function?

```
optim.binomial$par
```

```
## [1] -4.077575  1.504624
```

The graph displays the 'Pass' probability over 'Hours' for two categories: FALSE (red) and TRUE (teal). The x-axis represents 'Hours' from 0 to 5.5, and the y-axis represents 'Pass' probability from 0.00 to 1.00. A blue curve shows the overall trend, starting at 0.00 and approaching 1.00. Red dots and vertical lines indicate FALSE predictions, while teal dots and vertical lines indicate TRUE predictions.

Hours	Predict_Pass	Pass Probability
0.0	FALSE	0.00
0.2	FALSE	0.02
0.4	FALSE	0.03
0.6	FALSE	0.05
0.8	FALSE	0.08
1.0	FALSE	0.10
1.2	FALSE	0.12
1.4	FALSE	0.15
1.6	FALSE	0.20
1.8	FALSE	0.25
2.0	FALSE	0.30
2.2	FALSE	0.35
2.4	FALSE	0.40
2.6	TRUE	0.50
2.8	TRUE	0.60
3.0	TRUE	0.65
3.2	TRUE	0.70
3.4	TRUE	0.75
3.6	FALSE	0.80
3.8	TRUE	0.85
4.0	TRUE	0.90
4.2	TRUE	0.92
4.4	TRUE	0.94
4.6	TRUE	0.96
4.8	TRUE	0.97
5.0	TRUE	0.98
5.2	TRUE	0.99
5.4	TRUE	1.00

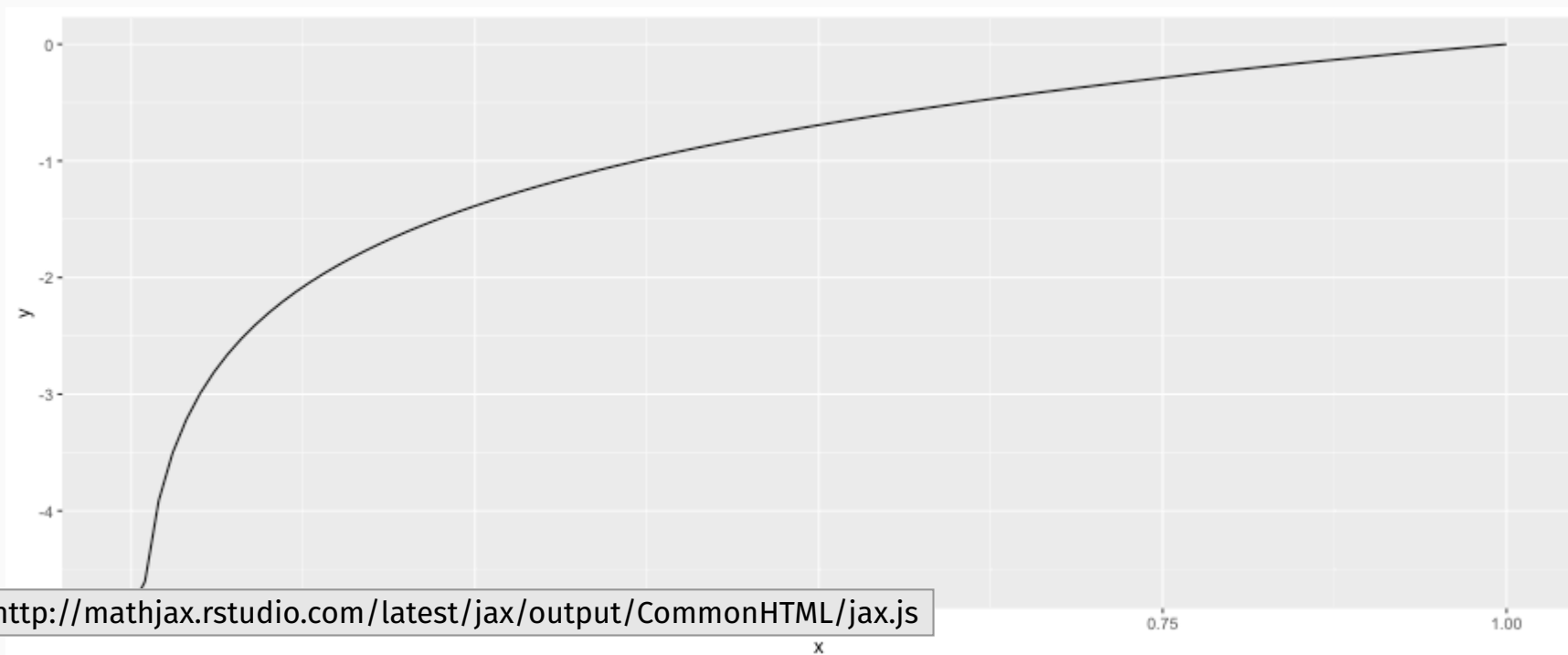


DAV 5300 Spring 2026

But why log likelihood?

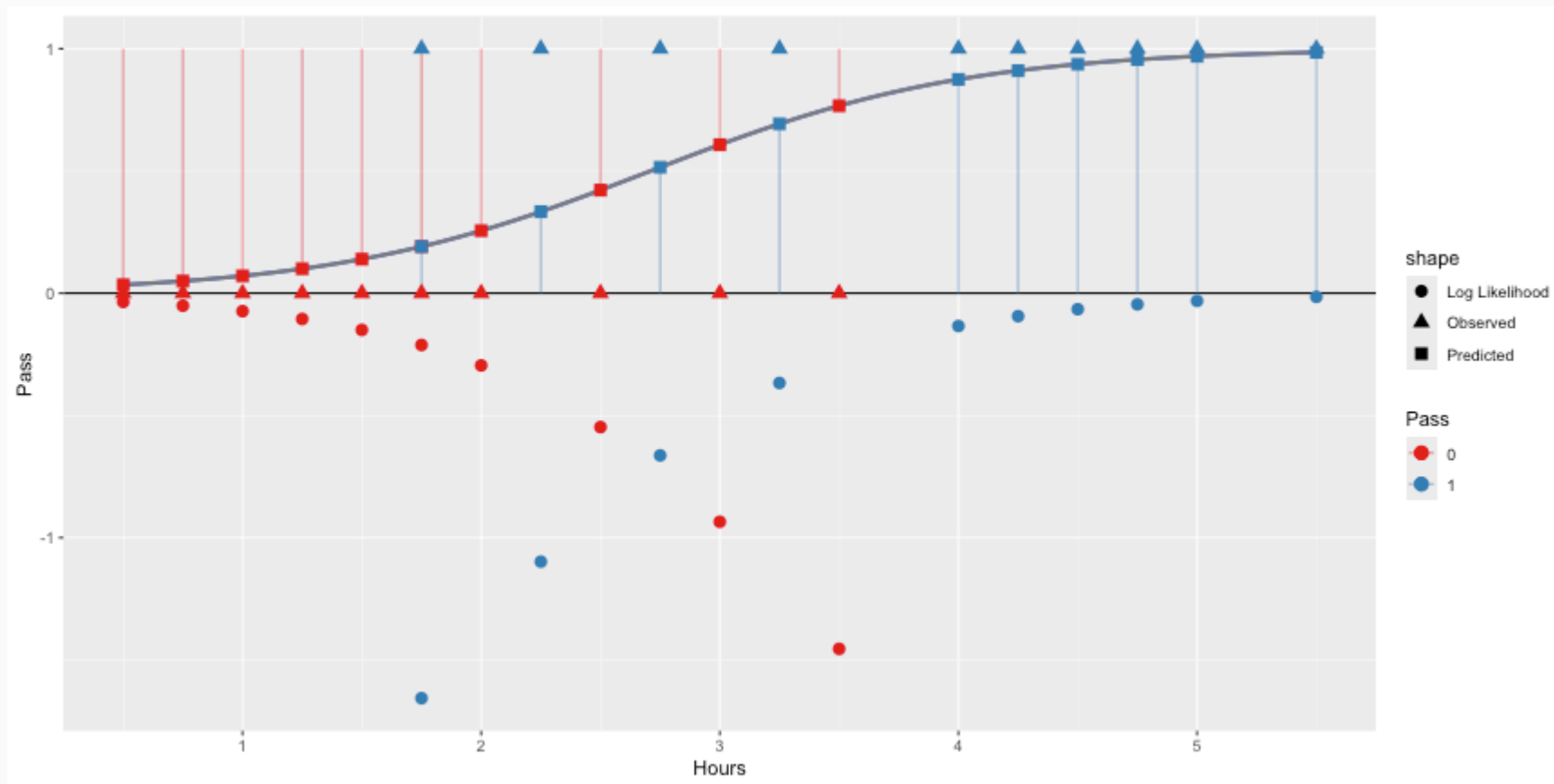
Since we know our outcomes are either zero or one, and hence bounded by zero and one, then we are only considering values of $\log(x)$ between zero and one. The plot below shows that $\log(x)$ for all $0 \leq x \leq 1$ is negative, going asymptotically to $-\infty$ as x approaches zero.

```
ggplot() + geom_function(fun = log) + xlim(0, 1)
```



File failed to load: <http://mathjax.rstudio.com/latest/jax/output/CommonHTML/jax.js>

But why log likelihood?



Assumptions

Although maximizing the log-likelihood provides a result similar to minimizing the sum of squared residuals using the logistic function, the log-likelihood doesn't rely on the assumptions of residuals OLS does.

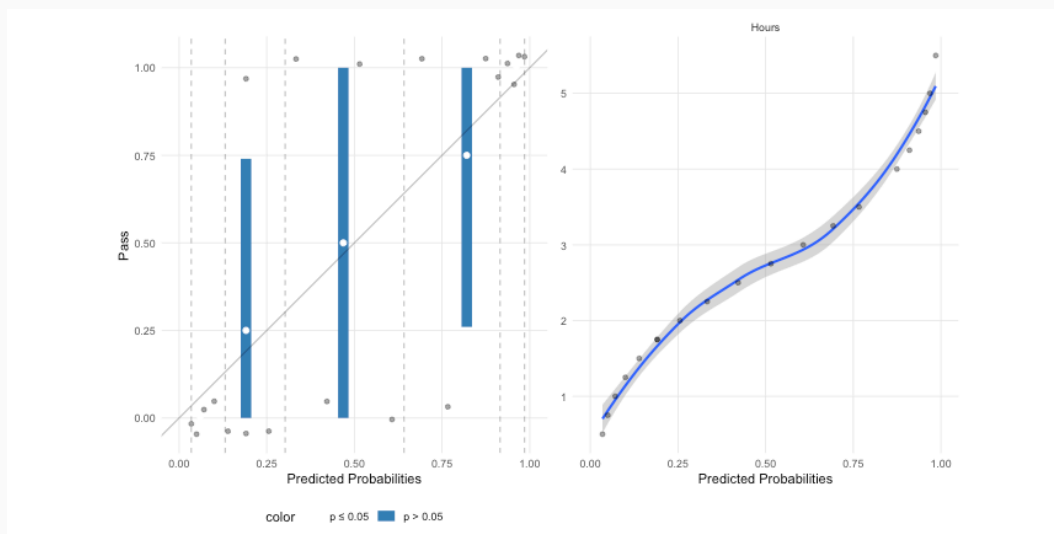
Namely:

- There is no assumption of linearity between the dependent and independent variables.
- Homoscedasticity (constant variance) is not for logistic regression (but is for linear regression).
- The residuals do not have to be normally distributed.

There is an assumption of linearity between the independent variable(s) and the log-

File failed to load: <http://mathjax.rstudio.com/latest/jax/output/CommonHTML/jax.js>
ODDS.

```
lr.out <- glm(Pass ~ Hours, data = study,  
              family = binomial(link='logit'))  
plot_linear_assumption_check(lr.out, n_groups = 5)
```

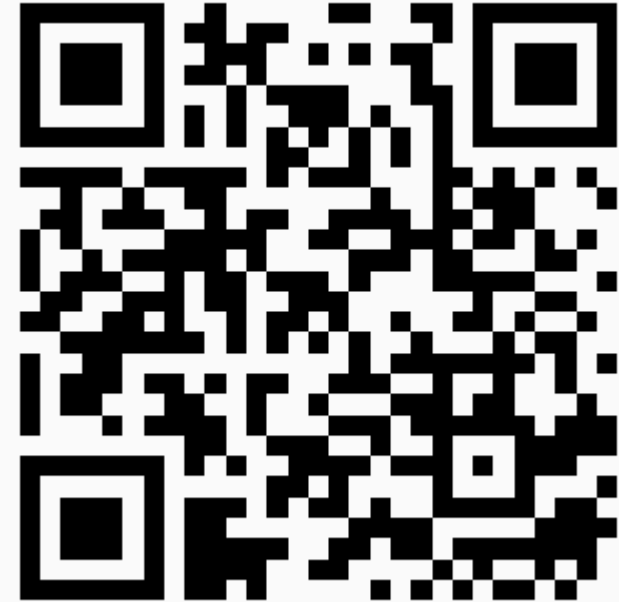


Additional Resources

- The Path to Log Likelihood
- Visual Introduction to Maximum Likelihood Estimation
- VisualStats R Package
- Logistic Regression Details Pt 2: Maximum Likelihood
- StatQuest: Maximum Likelihood, clearly explained
- Probability concepts explained: Maximum likelihood estimation

One Minute Paper

1. What was the most important thing you learned during this class?
2. What important question remains unanswered for you?



<https://forms.gle/hWUktVZ4Fyiia3xy6>